FreqBlender with Structural Secant Distillation (SSD)

Team No: [XX]

Members:
Sitanshu Mohapatra (230102120)
V Yavanash Sarma (230106067)
Gutti Shivighn (230103039)
Srayash Singh (230106069)
Chaitanya Kumar (230102120)

Github: https://github.com/Alter-Sitanshu/frequency_blender

1. Summary of the Assigned Paper (Maximum 100 Words)

Deepfake detectors often fail on new, unseen forgeries because they take a lazy shortcut: instead of learning what a face forgery actually looks like, they memorize harmless background grain or camera noise from the training set. To solve this, [**FreqBlender (NeurIPS 2024)**] treats an image like musical waves rather than raw pixels. Using a helper network (FPNet), it separates faces into low-frequency identity, mid-frequency structure (where fake blending edges hide), and high-frequency noise. By transplanting only the fake structural waves into real faces during training, it forces the detector to spot genuine forgery flaws rather than memorizing camera static.

2. Proposed Novelty and Technical Contribution

1. **Why We Need a New Approach (The Speed Bottleneck):** In the original [FreqBlender] paper, the frequency-blending helper network (FPNet) is only used during training to generate harder practice images; at test time, it is already discarded. Therefore, simply removing FPNet cannot make test-time detection any faster. To achieve a real speedup for mobile or real-time deployment, we replace the paper's heavy, slow classifier (**EfficientNet-B4**, the "Teacher" T, 17.55M parameters) with a tiny, fast network (**MobileNetV3-Small**, the "Student" S, 1.52M parameters).
2. **What We Introduce (Structural Secant Distillation – SSD):** Shrinking a network by 11.5× usually hurts accuracy, and standard [Knowledge Distillation (KD)] only asks the Student to copy the Teacher's final "Real vs. Fake" score on normal images—like copying a professor's multiple-choice answers without knowing *which clues* the professor looked at. Worse, if the heavy Teacher is under-trained on a limited compute budget, blindly copying its static guesses drags the Student down. We introduce **Structural Secant Distillation (SSD)**, a loss function that teaches the Student *how to react* when an image's frequency bands are slightly nudged.
3. **How SSD Differs from the Baseline and Prior Distillation:** Unlike the baseline (which only uses FPNet to mix training images) and prior frequency distillation ([ADD]; [Frequency Attention KD]), SSD uses frozen FPNet masks to poke the image in two ways during Student training: (a) a tiny **structural nudge** (injecting mid-frequency forgery waves), where the Student must match the Teacher's jump in suspicion, and (b) a tiny **noise nudge** (injecting high-frequency grain), where the Student is trained to stay unfazed whenever the Teacher ignores that noise. At test time, SSD adds **zero extra parameters or latency**.

3. Theoretical / Mathematical Justification

To understand why SSD works, we break its math down into five simple, common-sense steps:

Step 1 – Converting Images into Frequency Waves and Normalizing Stencils: First, the 2D Discrete Cosine Transform (DCT), written as $\phi(x)$, acts like a prism that turns an image x into a map of spatial frequencies, while ϕ^{-1} turns those frequencies cleanly back into pixels. In the baseline, FPNet predicts three soft "stencils" (masks) with values between 0 and 1: M_{sem} (low-frequency face identity, checked by [ArcFace]/[FaceNet]), M_{str} (mid-frequency edges and forgery flaws), and M_{noi} (high-frequency grain). To create blended training faces x' from a real face r and fake face f, the baseline grafts fake structure onto the real face:

$$x' = \phi^{-1}\Big[(M_{sem}(f) + M_{noi}(f)) \odot \phi(r) + M_{str}(f) \odot \phi(f)\Big] \tag{1}$$

where $\odot$ means multiplying two grids entry-by-entry. Because FPNet uses independent sigmoids, the three masks do not add up to exactly 1 on their own. During Student distillation, we divide each mask by their sum (plus a tiny $\varepsilon = 10^{-8}$ to prevent division by zero) so they form an exact 100% partition: $\bar{M}_k(x) = M_k(x)/\big(\sum_j M_j(x) + \varepsilon\big)$.

Step 2 – Measuring Raw Suspicion (The Logit Margin): Before converting outputs into percentages, a detector $h \in \{T, S\}$ outputs two raw scores (logits): $z_{h,1}(x)$ for "Fake" and $z_{h,0}(x)$ for "Real." Their difference is the **logit margin**:

$$a_h(x) = z_{h,1}(x) - z_{h,0}(x) \tag{2}$$

A larger positive $a_h(x)$ means higher suspicion that the image is Fake. We use raw margins instead of probabilities because probabilities flatten out near 100% and hide small changes.

Step 3 – Making a Controlled Surgical Nudge in Frequency Space: Given a base image x and a paired donor image d of the same person (from [Self-Blended Images (SBI)]), we isolate the difference in just one frequency band $k \in \{str, noi\}$ and convert it back to pixels:

$$\delta_k = \phi^{-1}\Big[\bar{M}_k(x) \odot (\phi(d) - \phi(x))\Big], \qquad g_k = \min\left(g_{max},\ \frac{\epsilon}{\max(\mathrm{RMS}(\delta_k), 10^{-8})}\right), \qquad x_k = \mathrm{clip}_{[0,1]}(x + \eta g_k \delta_k) \tag{3}$$

Why this formula is needed: In some image pairs, δ_k is huge; in others, it is tiny. Dividing $\epsilon = 0.02$ by the Root Mean Square average size $\mathrm{RMS}(\delta_k)$ and multiplying by $\eta = 0.25$ guarantees that our pixel nudge $\eta g_k \delta_k$ has an average size of at most $0.25 \times 0.02 = 0.005$ (0.5% of the $[0, 1]$ pixel range—invisible to the human eye). Meanwhile, capping the multiplier at $g_{max} = 4$ ensures that if δ_k is nearly zero, we do not multiply tiny computer rounding errors by infinity. Finally, $\mathrm{clip}_{[0,1]}$ keeps all pixel brightness values strictly between 0 (black) and 1 (white).

Step 4 – The Secant Slope and Why It Beats Calculus Derivatives: Next, we measure how fast the model's suspicion score a_h rises per unit of actual post-clipping pixel change $q_k = \mathrm{RMS}(x_k - x)$:

$$D_{h,k} = \frac{c\,[a_h(x_k) - a_h(x)]}{\max(q_k, q_{min})} \approx c\left\langle \nabla_x a_h(x),\ \frac{v_k}{\mathrm{RMS}(v_k)} \right\rangle, \qquad c = 0.01,\ q_{min} = 0.001,\ v_k = x_k - x \tag{4}$$

Theoretical Link (Secants vs. Jacobians): In basic calculus, the slope between two nearby points on a curve is called a **secant line** (rise/run). By a first-order Taylor approximation, when the step v_k is tiny, this secant slope $D_{h,k}$ equals the model's directional derivative (how sensitive its gradient $\nabla_x a_h(x)$ is along frequency band k). Why not compute the calculus derivative $\nabla_x a_h(x)$ directly? Because training the Student already requires taking derivatives with respect to its weights; putting a derivative inside the loss would require taking a *derivative of a derivative* (second-order backpropagation), which is slow and memory-heavy. The secant formula captures the exact same directional sensitivity using simple forward passes!

Step 5 – The Double-Gated SSD Loss and Total Objective: We train the Student using two smart safety switches—a **Correctness Gate** $w(x) = \max(0, 2p_T(y \mid x) - 1)$ and a **Noise Gate** $b(x) = \exp(-|D_{T,noi}|)$:

$$\mathcal{L}_{SSD} = \mathbb{E}_x\Big\{w(x)\,\mathrm{Huber}(D_{S,str} - D_{T,str}) + \gamma\, w(x)b(x)\,\mathrm{Huber}(D_{S,noi})\Big\} \tag{5}$$

$$\mathcal{L}_S = \mathrm{CE}(S(x), y) + \alpha\tau^2\mathrm{KL}(p_T^\tau(x)\|p_S^\tau(x)) + \beta\mathcal{L}_{SSD} \tag{6}$$

- **Matching Structural Instincts (**$\mathrm{Huber}(D_{S,str} - D_{T,str})$**):** If adding fake mid-frequency structure causes the Teacher's alarm to jump by +5, the Student's alarm must also jump by +5. (Huber/SmoothL1 loss squares small errors smoothly and treats large outliers linearly so one weird image cannot destabilize training.)
- **Conditional Noise Immunity (**$b(x)\,\mathrm{Huber}(D_{S,noi})$**):** Notice there is no $D_{T,noi}$ inside the second Huber term—we push the Student's noise reaction toward **zero**. However, what if high frequencies in a specific image actually contain useful edge clues that the Teacher relies on ($|D_{T,noi}|$ is large)? Then the exponential gate $b(x) = e^{-|D_{T,noi}|}$ drops toward 0, automatically turning off the noise penalty so we never blind the Student!
- **Ignoring Bad Teacher Advice (**$w(x)$**):** If the Teacher assigns less than 50% probability ($p_T(y \mid x) \le 0.5$) to the true label y, then $w(x) = 0$. This mutes the Teacher whenever it guesses wrong.
- **Implementation Safeguards:** We feed $[x, x_{str}, x_{noi}]$ in a single combined batch so Batch Normalization uses identical statistics across all three views, and disable 16-bit mixed precision (AMP) so 32-bit floats preserve the tiny 0.5% nudge responses.

4. Comparative Results and Empirical Evaluation

Experimental Setup: All six models are trained and evaluated on the [**FaceForensics++ (FF++)**] benchmark using official video splits, 224×224 face crops, Adam ($lr = 10^{-4}$), synthesis probability 0.2, and group-bootstrapped 95% confidence intervals (1,000 resamples).

Table 1: Quantitative comparison of Teacher baselines, Student baselines, and proposed SSD variants on **FaceForensics++ (FF++)**. Best overall values are in **bold**; best distilled student values are underlined.

Method / Configuration	AUC [95% CI] (↑)	Bal. Acc (↑)	Params (M ↓)	Latency (ms ↓)	Tput (img/s ↑)	Speedup (↑)
Teacher Spatial-Only (B4)	0.5668 [0.5509, 0.5833]	0.5489	17.55	14.37	247.9	1.01×
Teacher + FreqBlender (T)	0.5524 [0.5351, 0.5686]	0.5357	17.55	14.45	250.8	1.00×
Student + CE (Undistilled)	**0.6583** [0.6340, 0.6637]	0.5729	**1.52**	**4.63**	3,265.1	**3.12**×
Student + Standard KD	0.5733 [0.5590, 0.5890]	0.5576	**1.52**	4.98	**3,430.8**	2.90×
Student + SSD Structure (γ=0)	<u>0.6071</u> [0.5911, 0.6224]	**0.5788**	**1.52**	4.83	3,288.8	2.99×
Student + Full SSD (Ours)	0.5951 [0.5784, 0.6100]	0.5706	**1.52**	4.88	3,268.1	2.96×
Δ (**Full SSD** – Teacher T)	**+0.0426**	**+0.0349**	**−16.03**	**−9.57**	**+3,017.3**	**+1.96**×
Δ (**Full SSD** – Standard KD)	**+0.0217** [+0.0028, +0.0390]	**+0.0130**	0.00	−0.10	−162.7	+0.06×
Δ (**SSD Structure** – Standard KD)	**+0.0338**	**+0.0212**	0.00	−0.15	−142.0	+0.09×

Result Interpretation & Performance Trade-offs:

- **Massive Speed and Memory Gains Over the Baseline Teacher:** Replacing EfficientNet-B4 (17.55M params, 334.3 MB peak VRAM) with MobileNetV3-Small (1.52M params, 64.7 MB peak VRAM) shrinks model size by **11.5**× (−91.3%), cuts peak GPU memory by **5.2**×, speeds up single-image latency by $\sim$ **3**× (4.88 ms vs. 14.45 ms), and boosts batch throughput by **13**× (3,268 vs. 251 img/s).
- **Why Standard KD Hurts—and How SSD Rescues Distillation:** Under the short notebook training budget and 224×224 resolution, the heavy 17.55M-parameter Teacher is under-converged (0.5524 AUC), whereas the lightweight 1.52M-parameter Student converges faster on its own (`student_ce`: 0.6183 AUC). Consequently, forcing the Student to blindly copy the weaker Teacher's static probabilities (`student_kd`) drags Student AUC down to 0.5733. Adding our **Full SSD** objective recovers **+0.0217** AUC over Standard KD (with a 1,000-resample paired bootstrap 95% CI of [+0.0028, +0.0390], proving statistical significance since the entire interval is above zero) and beats the Teacher by **+0.0426** AUC.
- **Ablation Insight (Structure-Only vs. Full SSD):** Isolating the structural secant term (`student_structure`, $\gamma = 0$) achieves **0.6071** AUC (+0.0338 over KD) and the **highest Balanced Accuracy of all six models (0.5788)**, even surpassing `student_ce` (0.5729). Adding the noise penalty ($\gamma = 0.25$ in `student_proposed`) slightly lowers in-domain AUC from 0.6071 to 0.5951 (and slightly increases p95 tail latency from 5.15 ms to 7.34 ms due to timing variance) because with an under-trained Teacher, $|D_{T,noi}|$ stays small ($b(x) \approx 1$), over-penalizing high-frequency cues that still carry helpful edge information at 224×224 resolution.